

```
In [1]: import warnings
warnings.filterwarnings("ignore")
```

Data Preprocessing:

The text dataset is processed and transformed, to use it for model training. Steps:

- Loading Dataset
- Initialising a tokenizer object for the pretrained model of BERT Transformer
- The transformations on dataset involve
 - Encoding the words with their respective word 'id' from the BERT Model
 - The sentences are padded to make all the sentences of same length in our case the `max_length = 200`
 - Respective attention vectors are created for each sentence with value '1' for word and value '0' for padded words.
- This transformed requires to have a type of tensorflow tensor data. So we will transform the data into Tensorflow tensor data format.

```
In [2]: from datasets import load_dataset

dataset = load_dataset("ag_news")
split = dataset['train'].train_test_split(test_size=0.1, seed=42)
train_ds = split["train"]
val_ds = split['test']
test_ds = dataset['test']
```

```
In [3]: from transformers import AutoTokenizer

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
In [4]: def encode_batch(batch):
    return tokenizer(
        batch['text'],
        max_length = 200,
        truncation = True,
        padding = "max_length"
    )

train_enc = train_ds.map(encode_batch, batched=True)
test_enc = test_ds.map(encode_batch, batched=True)
val_enc = val_ds.map(encode_batch, batched=True)

train_enc = train_enc.rename_column("label", "labels")
test_enc = test_enc.rename_column("label", "labels")
val_enc = val_enc.rename_column("label", "labels")
```

```
train_enc.set_format(type = 'tensorflow', columns = ['input_ids','attention_mask'],
test_enc.set_format(type = 'tensorflow', columns = ['input_ids','attention_mask', '
val_enc.set_format(type = 'tensorflow', columns = ['input_ids','attention_mask', 'l
```

```
Map: 100%|████████████████████████████████████████████████████████████████████████████████| 12000/12000
0 [00:04<00:00, 2733.28 examples/s]
```

```
In [5]: import tensorflow as tf

tf_train = train_enc.to_tf_dataset(
    columns=["input_ids", "attention_mask"],
    label_cols=["labels"],
    shuffle=True,
    batch_size=16,
)

tf_val = val_enc.to_tf_dataset(
    columns=["input_ids", "attention_mask"],
    label_cols=["labels"],
    shuffle=False,
    batch_size=16,
)

tf_test = test_enc.to_tf_dataset(
    columns=["input_ids", "attention_mask"],
    label_cols=["labels"],
    shuffle=False,
    batch_size=16,
)
```

Model Initialization

For the text classification task, we employ the BERT Base Uncased model, which is a pretrained Transformer-based encoder. BERT is a bidirectional autoencoder architecture trained on large-scale corpora using two objectives:

- Masked Language Modeling (MLM) — learns deep contextual word representations
- Next Sentence Prediction (NSP) — learns relationships between sentences

Because of this pretraining, BERT produces rich semantic embeddings for every token and for the entire sentence through the [CLS] embedding.

How BERT is used in our classification task

For sequence (text) classification, we follow the standard fine-tuning setup:

- The input text is tokenized into word-piece tokens and converted to integer IDs.
- These token IDs and their attention masks are passed into BERT's encoder stack.

- BERT produces a contextual embedding for every token and a special [CLS] embedding, which summarizes the meaning of the entire input sequence.

On top of this [CLS] embedding, we place a fully connected classification layer:

$$\mathbf{h}_{\text{CLS}} \in \mathbb{R}^{768} \rightarrow \text{Dense}(4)$$

where 4 corresponds to the four AG News categories.

This classification layer is randomly initialized, and only its weights are trained from scratch, while BERT's encoder weights are fine-tuned during training to adapt the representations to our dataset.

What the model learns

- BERT provides context-aware embeddings that capture syntax, semantics, and long-range dependencies.
- The classification head learns to map these embeddings to a 4-class probability distribution using a softmax activation.
- Training optimizes the cross-entropy loss between predicted and true labels.

```
In [6]: from transformers import AutoTokenizer, TFBertModel

model_name = "bert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)

bert = TFBertModel.from_pretrained(
    model_name,
    from_pt=True # IMPORTANT: Loads PyTorch weights into TF
)
```

TensorFlow and JAX classes are deprecated and will be removed in Transformers v5. We recommend migrating to PyTorch classes or pinning your version of Transformers.

Some weights of the PyTorch model were not used when initializing the TF 2.0 model TFBertModel: ['cls.predictions.transform.LayerNorm.weight', 'cls.predictions.transform.dense.weight', 'cls.seq_relationship.bias', 'cls.seq_relationship.weight', 'cls.predictions.transform.LayerNorm.bias', 'cls.predictions.decoder.weight', 'cls.predictions.transform.dense.bias', 'cls.predictions.bias']

- This IS expected if you are initializing TFBertModel from a PyTorch model trained on another task or with another architecture (e.g. initializing a TFBertForSequenceClassification model from a BertForPreTraining model).
- This IS NOT expected if you are initializing TFBertModel from a PyTorch model that you expect to be exactly identical (e.g. initializing a TFBertForSequenceClassification model from a BertForSequenceClassification model).

All the weights of TFBertModel were initialized from the PyTorch model.

If your task is similar to the task the model of the checkpoint was trained on, you can already use TFBertModel for predictions without further training.

```
In [7]: import tensorflow as tf
from tensorflow.keras.layers import Input, Dense, Dropout
from tensorflow.keras.models import Model

max_len = 200

input_ids = Input(shape=(max_len,), dtype=tf.int32, name="input_ids")
attention_mask = Input(shape=(max_len,), dtype=tf.int32, name="attention_mask")
```

```
In [8]: bert_outputs = bert(
    input_ids=input_ids,
    attention_mask=attention_mask
)

cls_emb = bert_outputs.last_hidden_state[:, 0, :] # CLS token

x = Dropout(0.2)(cls_emb)
x = Dense(256, activation="relu")(x)
x = Dropout(0.2)(x)
out = Dense(4, activation="softmax")(x)

model = Model(
    inputs=[input_ids, attention_mask],
    outputs=out
)

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=2e-5),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
In [9]: model.summary()
```

Model: "model"

Layer (type)	Output Shape	Param #	Connected to
=====			
input_ids (InputLayer)	[(None, 200)]	0	[]
attention_mask (InputLayer)	[(None, 200)]	0	[]
tf_bert_model (TFBertModel)	TFBaseModelOutputWithPoolingAndCrossAttentions(last_hidden_state=(None, 200, 768), pooler_output=(None, 768), past_key_values=None, hidden_states=None, attentions=None, cross_attentions=None)	109482240	['input_ids[0][0]', 'attention_mask[0][0]']
tf.__operators__.getitem (SlicingOpLambda)	(None, 768)	0	['tf_bert_model[0][0]']
dropout_37 (Dropout)	(None, 768)	0	['tf.__operators__.getitem[0][0]']
dense (Dense)	(None, 256)	196864	['dropout_37[0][0]']
dropout_38 (Dropout)	(None, 256)	0	['dense[0][0]']
dense_1 (Dense)	(None, 4)	1028	['dropout_38[0][0]']
=====			
Total params: 109,680,132			
Trainable params: 109,680,132			
Non-trainable params: 0			

```
In [10]: history = model.fit(
          tf_train,
          validation_data = tf_val,
          epochs = 10,
          )
```

Epoch 1/10

WARNING:tensorflow:Gradients do not exist for variables ['tf_bert_model/bert/pooler/dense/kernel:0', 'tf_bert_model/bert/pooler/dense/bias:0'] when minimizing the loss. If you're using `model.compile()`, did you forget to provide a `loss` argument?

WARNING:tensorflow:Gradients do not exist for variables ['tf_bert_model/bert/pooler/dense/kernel:0', 'tf_bert_model/bert/pooler/dense/bias:0'] when minimizing the loss. If you're using `model.compile()`, did you forget to provide a `loss` argument?

140/6750 [.....] - ETA: 46:26:11 - loss: 0.5672 - accuracy: 0.7848

KeyboardInterrupt

Traceback (most recent call last)

Cell In[10], line 1

```

----> 1 history = model.fit(
      2     tf_train,
      3     validation_data = tf_val,
      4     epochs = 10,
      5 )

```

File ~\anaconda3\envs\cuda-env\lib\site-packages\keras\utils\traceback_utils.py:65, in filter_traceback.<locals>.error_handler(*args, **kwargs)

```

63 filtered_tb = None
64 try:
--> 65     return fn(*args, **kwargs)
66 except Exception as e:
67     filtered_tb = _process_traceback_frames(e.__traceback__)

```

File ~\anaconda3\envs\cuda-env\lib\site-packages\keras\engine\training.py:1564, in Model.fit(self, x, y, batch_size, epochs, verbose, callbacks, validation_split, validation_data, shuffle, class_weight, sample_weight, initial_epoch, steps_per_epoch, validation_steps, validation_batch_size, validation_freq, max_queue_size, workers, use_multiprocessing)

```

1556 with tf.profiler.experimental.Trace(
1557     "train",
1558     epoch_num=epoch,
1559     ...)
1561     _r=1,
1562 ):
1563     callbacks.on_train_batch_begin(step)
-> 1564     tmp_logs = self.train_function(iterator)
1565     if data_handler.should_sync:
1566         context.async_wait()

```

File ~\anaconda3\envs\cuda-env\lib\site-packages\tensorflow\python\util\traceback_utils.py:150, in filter_traceback.<locals>.error_handler(*args, **kwargs)

```

148 filtered_tb = None
149 try:
--> 150     return fn(*args, **kwargs)
151 except Exception as e:
152     filtered_tb = _process_traceback_frames(e.__traceback__)

```

File ~\anaconda3\envs\cuda-env\lib\site-packages\tensorflow\python\eager\def_function.py:915, in Function.__call__(self, *args, **kws)

```

912 compiler = "xla" if self._jit_compile else "nonXla"
914 with OptionalXlaContext(self._jit_compile):
--> 915     result = self._call(*args, **kws)
917 new_tracing_count = self.experimental_get_tracing_count()
918 without_tracing = (tracing_count == new_tracing_count)

```

File ~\anaconda3\envs\cuda-env\lib\site-packages\tensorflow\python\eager\def_function.py:947, in Function._call(self, *args, **kws)

```

944 self._lock.release()
945 # In this case we have created variables on the first call, so we run the
946 # defunned version which is guaranteed to never create variables.
--> 947 return self._stateless_fn(*args, **kws) # pylint: disable=not-callable
948 elif self._stateful_fn is not None:

```

```

949 # Release the lock early so that multiple threads can perform the call
950 # in parallel.
951 self._lock.release()

```

File ~\anaconda3\envs\cuda-env\lib\site-packages\tensorflow\python\eager\function.py:2496, in Function.__call__(self, *args, **kwargs)

```

2493 with self._lock:
2494     (graph_function,
2495      filtered_flat_args) = self._maybe_define_function(args, kwargs)
-> 2496 return graph_function._call_flat(
2497     filtered_flat_args, captured_inputs=graph_function.captured_inputs)

```

File ~\anaconda3\envs\cuda-env\lib\site-packages\tensorflow\python\eager\function.py:1862, in ConcreteFunction._call_flat(self, args, captured_inputs, cancellation_manager)

```

1858 possible_gradient_type = gradients_util.PossibleTapeGradientTypes(args)
1859 if (possible_gradient_type == gradients_util.POSSIBLE_GRADIENT_TYPES_NONE
1860     and executing_eagerly):
1861     # No tape is watching; skip to running the function.
-> 1862 return self._build_call_outputs(self._inference_function.call(
1863     ctx, args, cancellation_manager=cancellation_manager))
1864 forward_backward = self._select_forward_and_backward_functions(
1865     args,
1866     possible_gradient_type,
1867     executing_eagerly)
1868 forward_function, args_with_tangents = forward_backward.forward()

```

File ~\anaconda3\envs\cuda-env\lib\site-packages\tensorflow\python\eager\function.py:499, in _EagerDefinedFunction.call(self, ctx, args, cancellation_manager)

```

497 with _InterpolateFunctionError(self):
498     if cancellation_manager is None:
--> 499     outputs = execute.execute(
500         str(self.signature.name),
501         num_outputs=self._num_outputs,
502         inputs=args,
503         attrs=attrs,
504         ctx=ctx)
505     else:
506     outputs = execute.execute_with_cancellation(
507         str(self.signature.name),
508         num_outputs=self._num_outputs,
509         ...)
511     ctx=ctx,
512     cancellation_manager=cancellation_manager)

```

File ~\anaconda3\envs\cuda-env\lib\site-packages\tensorflow\python\eager\execute.py:54, in quick_execute(op_name, num_outputs, inputs, attrs, ctx, name)

```

52 try:
53     ctx.ensure_initialized()
---> 54     tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name,
55         inputs, attrs, num_outputs)
56 except core._NotOkStatusException as e:
57     if name is not None:

```

KeyboardInterrupt:

In []: